

Depth-First Search (DFS)

Algorithm Design → Search

Discrete Maths for Engineers

Leandro Rodrigues

March 11, 2026

Contents

1	Graph Model & Problem Setting	3
1.1	Graph Model	3
1.2	The Exploration Problem	3
1.3	Structure of the Exploration	3
1.4	Reachability	3
1.5	Why DFS Matters	3
2	The Core Idea (Deep Exploration + Backtracking)	4
2.1	The Central Idea	4
2.2	Deep Exploration	4
2.3	Backtracking	4
2.4	The Implicit Stack	4
2.5	Visual Intuition	4
3	The Algorithm (CLRS Pseudocode)	5
3.1	The Algorithm	5
3.2	State Maintained by DFS	5
3.3	The Meaning of Colors	5
3.4	Main Procedure — DFS	6
3.5	Recursive Procedure — DFS-VISIT	6
3.6	Discovery and Finish Times	6
3.7	Parent Pointers and DFS Trees	7
3.8	Why DFS May Produce a Forest	7
4	Structural Properties & Discovery Times	8
4.1	Structural Properties & Discovery Times	8
4.2	Discovery and Finish Times	8
4.3	The Interval View	8
4.4	Parenthesis Theorem	8
4.5	Nesting Example	9
4.6	Ancestor–Descendant Criterion	9
4.7	Disjoint Intervals and Separate Branches	9
4.8	White-Path Intuition	9
4.9	Why These Properties Matter	9

5	Edge Classification	10
5.1	Edge Classification	10
5.2	Why Edge Classification Matters	10
5.3	The Four Edge Types	10
5.4	Tree Edges	11
5.5	Back Edges	11
5.6	Forward Edges	11
5.7	Cross Edges	11
5.8	Characterization by Timestamps	11
5.9	Special Case: Undirected Graphs	12
5.10	Consequence for DAGs	12
5.11	Mini Summary	12
6	Applications I (Cycles & Topological Sorting)	13
6.1	Applications I	13
6.2	Why DFS Detects Cycles	13
6.3	Back Edges and Directed Cycles	13
6.4	Operational Test Using Colors	13
6.5	Cycle Detection Sketch	14
6.6	DAGs	14
6.7	Topological Sorting — The Core Idea	14
6.8	Finish Times and Ordering	14
6.9	DFS-Based Topological Sort	15
6.10	Engineering Use Cases	15
6.11	Mini Summary	15
7	Applications II (SCC & Advanced Structures)	16
7.1	Applications II	16
7.2	Strongly Connected Components	16
7.3	SCCs as a Higher-Level Structure	16
7.4	Why Finish Times Matter Again	16
7.5	The Transpose Graph	17
7.6	Kosaraju’s Algorithm	17
7.7	Kosaraju — Pseudocode Sketch	17
7.8	Why Kosaraju Works	17
7.9	Complexity	17
7.10	Advanced DFS-Based Structures	18
7.11	Engineering Use Cases	18
8	DFS vs BFS	19
8.1	DFS vs BFS	19
8.2	The Core Difference	19
8.3	Queue vs Stack	19
8.4	What BFS Naturally Computes	20
8.5	What DFS Naturally Computes	20
8.6	Side-by-Side Comparison	20
8.7	When to Prefer DFS	21
8.8	When to Prefer BFS	21
8.9	Complexity	21
8.10	Common Misconceptions	21
8.11	Series Summary	21

1 Graph Model & Problem Setting

1.1 Graph Model

Depth-First Search (DFS) operates on a graph formally defined as

$$G = (V, E)$$

- V — the set of vertices
- E — the set of edges

Graphs may be:

- Directed
- Undirected

DFS works in both cases and allows the algorithm to explore the structure of the graph starting from a given vertex.

1.2 The Exploration Problem

Given a graph $G = (V, E)$ and a starting vertex $s \in V$, the goal is to explore all vertices reachable from s .

DFS performs this exploration by following a path as deep as possible before returning and exploring alternative paths.

Key idea: explore deeply first, then backtrack.

1.3 Structure of the Exploration

DFS explores the graph by following a single path until no further unvisited vertices can be reached.

When that happens, the algorithm returns to the previous vertex and continues exploring other unexplored edges.

This behavior is known as **backtracking**.

1.4 Reachability

A vertex v is said to be reachable from s if there exists a path from s to v .

DFS discovers exactly the set of vertices reachable from the source.

This property makes DFS useful for connectivity analysis in graphs.

1.5 Why DFS Matters

Depth-First Search is one of the fundamental graph traversal algorithms.

It serves as the foundation for several important algorithms in computer science.

- Cycle detection
- Topological sorting in DAGs
- Strongly connected components
- Structural analysis of graphs

2 The Core Idea (Deep Exploration + Backtracking)

2.1 The Central Idea

Depth-First Search explores a graph by following one path as far as possible before exploring alternative paths.

Starting from a source vertex s , the algorithm repeatedly moves to an unvisited neighbor until it reaches a vertex that has no unexplored edges.

Key idea: go deep first, then backtrack.

2.2 Deep Exploration

DFS expands the search by always selecting a neighbor that has not yet been visited.

This produces long exploration chains that follow a single branch of the graph.

Only when no further expansion is possible does the algorithm return to the previous vertex and continue exploring.

2.3 Backtracking

When a vertex has no unvisited neighbors, DFS performs backtracking.

The algorithm returns to the most recent vertex that still has unexplored edges.

Backtracking ensures that every reachable vertex in the graph is eventually explored.

2.4 The Implicit Stack

The exploration process naturally forms a stack structure.

- Each recursive call pushes a vertex onto the stack
- Returning from recursion pops the vertex

DFS can therefore be implemented either with recursion or with an explicit stack.

2.5 Visual Intuition

The traversal behaves like exploring a maze.

- Choose a direction
- Walk forward until you hit a dead end
- Go back to the last intersection
- Try another direction

This strategy systematically explores every reachable path.

3 The Algorithm (CLRS Pseudocode)

3.1 The Algorithm

Roadmap (what you will learn in this part):

- The state maintained by DFS
- The role of colors and parent pointers
- The global DFS procedure
- The recursive DFS-VISIT procedure
- Discovery and finish times
- Why DFS may produce a forest

In this part, we formalize Depth-First Search using the classical presentation from *Introduction to Algorithms* (CLRS).

DFS is not just an intuition about “going deep”. It is a precise graph traversal procedure with well-defined state, recursion structure, and timestamps.

3.2 State Maintained by DFS

For each vertex $u \in V$, DFS stores:

- $color[u]$ — exploration state
- $\pi[u]$ — predecessor (parent) in the DFS tree
- $d[u]$ — discovery time
- $f[u]$ — finish time

These values allow DFS to do much more than just traverse the graph. They encode structural information that will be essential later for edge classification, cycle detection, and topological sorting.

3.3 The Meaning of Colors

Color	Meaning
WHITE	The vertex has not been discovered yet.
GRAY	The vertex has been discovered but is still being explored.
BLACK	The vertex and all its reachable descendants have been fully explored.

Interpretation: GRAY means “currently active in the recursion stack”.

3.4 Main Procedure — DFS

The global DFS procedure initializes all vertices and then scans the graph. If it finds a WHITE vertex, it starts a new DFS tree rooted at that vertex.

```
DFS(G):  
  
    for each vertex u in V[G]:  
        color[u] = WHITE  
        [u] = NIL  
  
    time = 0  
  
    for each vertex u in V[G]:  
        if color[u] == WHITE:  
            DFS-VISIT(G, u)
```

Important: If the graph is disconnected, DFS will produce a DFS forest, not just a single tree.

3.5 Recursive Procedure — DFS-VISIT

The real exploration happens inside the recursive subroutine:

```
DFS-VISIT(G, u):  
  
    time = time + 1  
    d[u] = time  
    color[u] = GRAY  
  
    for each v in Adj[u]:  
        if color[v] == WHITE:  
            [v] = u  
            DFS-VISIT(G, v)  
  
    color[u] = BLACK  
    time = time + 1  
    f[u] = time
```

This procedure captures the full deep-exploration logic: discover a vertex, recursively visit unvisited neighbors, and only then finish the current vertex.

3.6 Discovery and Finish Times

DFS assigns two timestamps to every vertex:

- $d[u]$ — when u is first discovered
- $f[u]$ — when the exploration of u is complete

These timestamps are one of the most important features of DFS. They turn a traversal into a structural analysis tool.

Later, the interval $[d[u], f[u]]$ will let us reason about ancestry, nesting, and edge types.

3.7 Parent Pointers and DFS Trees

Whenever DFS discovers a WHITE vertex v from a vertex u , it sets:

$$\pi[v] = u$$

These parent pointers define the DFS tree rooted at the vertex that started the exploration.

If the graph has multiple disconnected regions, DFS builds a collection of trees, called a DFS forest.

3.8 Why DFS May Produce a Forest

DFS starts a new recursive exploration every time it encounters a vertex that is still WHITE in the outer loop.

Therefore:

- If the graph is fully reachable from a single source, DFS produces one tree.
- If the graph has multiple disconnected components, DFS produces multiple trees.

Conclusion: the complete DFS output is, in general, a forest.

4 Structural Properties & Discovery Times

4.1 Structural Properties & Discovery Times

Roadmap (what you will learn in this part):

- What discovery and finish times mean
- Why DFS intervals are structurally important
- The parenthesis theorem
- The white-path theorem intuition
- Ancestor–descendant relations in DFS trees
- Why these properties power later applications

Depth-First Search is more than a traversal rule. Its timestamps encode a hidden structure in the graph exploration.

In this part, we study the formal properties of discovery times, finish times, and the interval structure generated by recursion.

4.2 Discovery and Finish Times

During DFS, every vertex u receives two timestamps:

- $d[u]$ — the moment u is first discovered
- $f[u]$ — the moment the exploration of u is completed

These times satisfy:

$$d[u] < f[u]$$

for every vertex u , because a vertex must be discovered before it can be finished.

4.3 The Interval View

Each vertex defines a time interval:

$$[d[u], f[u]]$$

This interval represents the full active lifetime of u in the recursion.

Interpretation: a vertex is GRAY exactly while its interval is open.

The key structural fact is that DFS intervals are not arbitrary: they are either nested or disjoint.

4.4 Parenthesis Theorem

For any two vertices u and v , exactly one of the following holds:

- The intervals $[d[u], f[u]]$ and $[d[v], f[v]]$ are disjoint
- $[d[u], f[u]]$ is fully contained in $[d[v], f[v]]$
- $[d[v], f[v]]$ is fully contained in $[d[u], f[u]]$

Why “parenthesis”? DFS behaves like properly nested parentheses: once you open a recursive call, you must close it before closing its caller.

4.5 Nesting Example

Suppose DFS discovers u , then from u discovers v .

Then the timestamps must satisfy:

$$d[u] < d[v] < f[v] < f[u]$$

So the interval of v is nested inside the interval of u .

Conclusion: descendants live entirely inside the time interval of their ancestors.

4.6 Ancestor–Descendant Criterion

In a DFS tree, vertex u is an ancestor of v if and only if:

$$d[u] < d[v] < f[v] < f[u]$$

Equivalently, the interval of v is properly nested inside the interval of u .

This gives DFS a powerful structural test: ancestry can be read directly from timestamps.

4.7 Disjoint Intervals and Separate Branches

If two vertices lie in different DFS branches, or in different DFS trees of the forest, their intervals are disjoint.

So DFS timestamps separate the graph into nested recursive regions.

Structural insight: nesting means ancestry; disjointness means separation.

4.8 White-Path Intuition

A central theorem in DFS theory states: a vertex v becomes a descendant of u in the DFS forest if and only if, at the moment u is discovered, there exists a path from u to v consisting entirely of WHITE vertices.

Intuition: DFS can only recurse through vertices that have not yet been discovered.

This theorem helps justify why DFS trees encode reachability structure in such a precise way.

4.9 Why These Properties Matter

Discovery and finish times are not decorative metadata. They are the reason DFS supports:

- edge classification
- cycle detection
- topological sorting
- strongly connected components

Takeaway: DFS turns recursive exploration into a formal structural language.

5 Edge Classification

5.1 Edge Classification

Roadmap (what you will learn in this part):

- Why DFS classifies edges
- The four edge types
- Tree edges and DFS structure
- Back edges and cycle detection
- Forward and cross edges
- How timestamps reveal edge types
- Why this matters for DAGs and graph analysis

Once DFS timestamps are available, edges are no longer just connections in the graph. They acquire a structural meaning relative to the DFS forest.

In this part, we classify edges according to how they connect vertices in the recursive exploration.

5.2 Why Edge Classification Matters

DFS assigns discovery and finish times to vertices, and these timestamps allow us to interpret each edge in relation to the DFS forest.

This classification is fundamental for:

- cycle detection
- topological sorting
- reasoning about DAGs
- structural analysis of directed graphs

Key idea: edge classification turns DFS into a tool for graph theory, not just traversal.

5.3 The Four Edge Types

In a directed graph, every edge (u, v) encountered during DFS falls into one of four categories:

Type	Meaning
Tree edge	The edge discovers a new WHITE vertex.
Back edge	The edge points to an ancestor in the DFS tree.
Forward edge	The edge points to a proper descendant that was not discovered by this edge.
Cross edge	The edge connects unrelated DFS branches or different DFS trees.

5.4 Tree Edges

An edge (u, v) is a tree edge if v is WHITE when the edge is explored.

In that case, DFS sets:

$$\pi[v] = u$$

and the edge becomes part of the DFS tree (or DFS forest).

Tree edges define the recursive skeleton of the traversal.

5.5 Back Edges

An edge (u, v) is a back edge if it points from a vertex to one of its ancestors in the DFS tree.

Operationally, this often appears when v is GRAY at the moment the edge is examined.

Critical fact: a back edge indicates a cycle in a directed graph.

This is why DFS is the classical tool for cycle detection.

5.6 Forward Edges

An edge (u, v) is a forward edge if it points from a vertex to a proper descendant in the DFS tree, but the edge itself was not used to discover that descendant.

So the ancestor–descendant relation exists, but the edge is not a tree edge.

Forward edges appear only in directed graphs.

5.7 Cross Edges

An edge (u, v) is a cross edge if it connects vertices that are not in an ancestor–descendant relation.

Cross edges may connect:

- different branches of the same DFS tree
- different trees in the DFS forest

Cross edges also appear only in directed graphs.

5.8 Characterization by Timestamps

DFS timestamps let us reason about edge types formally.

If v is a descendant of u , then:

$$d[u] < d[v] < f[v] < f[u]$$

If the intervals are disjoint, the vertices are in separate branches:

$$f[v] < d[u] \quad \text{or} \quad f[u] < d[v]$$

The interval structure from Part 4 is exactly what makes edge classification possible.

5.9 Special Case: Undirected Graphs

In undirected graphs, DFS edge classification is simpler.

Every edge is either:

- a tree edge, or
- a back edge

There are no forward or cross edges in the undirected case.

This simplification is one reason DFS is especially elegant in undirected graph theory.

5.10 Consequence for DAGs

A directed graph is acyclic if and only if DFS finds no back edges.

DAG criterion: no back edge $\Leftrightarrow$ no directed cycle.

This result is one of the most important consequences of DFS edge classification, and it leads directly to topological sorting.

5.11 Mini Summary

- Tree edges discover new vertices.
- Back edges point to ancestors and indicate cycles.
- Forward edges point to descendants.
- Cross edges connect separate branches.
- In undirected graphs, only tree edges and back edges occur.

6 Applications I (Cycles & Topological Sorting)

6.1 Applications I

Roadmap (what you will learn in this part):

- Why DFS is a natural tool for cycle detection
- How back edges characterize directed cycles
- How to test whether a graph is a DAG
- Why finish times suggest an ordering
- How topological sorting is built from DFS
- Why topological sorting works only for DAGs
- Engineering use cases for dependency ordering

Depth-First Search is not only a traversal method. It is a structural tool that powers two classical applications: cycle detection and topological sorting.

In this part, we use the edge classification from Part 5 to solve concrete problems on directed graphs.

6.2 Why DFS Detects Cycles

DFS explores the graph recursively and keeps track of vertices that are currently active in the recursion stack.

If, during exploration, DFS reaches a vertex that is already active, then the search has returned to an ancestor.

Structural meaning: returning to an ancestor means a directed cycle exists.

6.3 Back Edges and Directed Cycles

From Part 5, a back edge is an edge from a vertex to one of its ancestors in the DFS tree.

Therefore:

$$G \text{ has a directed cycle} \iff \text{DFS finds a back edge}$$

Cycle criterion: a directed graph is acyclic if and only if DFS encounters no back edge.

6.4 Operational Test Using Colors

In practice, cycle detection is often implemented using DFS colors:

- WHITE — not discovered
- GRAY — currently active in recursion
- BLACK — fully explored

If DFS explores an edge (u, v) and finds v already GRAY, then (u, v) is a back edge, so a cycle exists.

Implementation rule: GRAY → cycle evidence in directed graphs.

6.5 Cycle Detection Sketch

A DFS-based cycle detector can be written as follows:

```
HAS-CYCLE(G):

    for each vertex u in V[G]:
        color[u] = WHITE

    for each vertex u in V[G]:
        if color[u] == WHITE:
            if DFS-CYCLE-VISIT(G, u):
                return true

    return false

DFS-CYCLE-VISIT(G, u):

    color[u] = GRAY

    for each v in Adj[u]:
        if color[v] == GRAY:
            return true
        if color[v] == WHITE:
            if DFS-CYCLE-VISIT(G, v):
                return true

    color[u] = BLACK
    return false
```

The algorithm stops as soon as it finds a back edge.

6.6 DAGs

A DAG is a Directed Acyclic Graph.

By the previous result:

$$G \text{ is a DAG} \iff \text{DFS finds no back edge}$$

DAGs are central in computer science because they model systems with dependencies but without circular requirements.

Examples: build pipelines, prerequisite graphs, task scheduling, dependency graphs.

6.7 Topological Sorting — The Core Idea

A topological ordering of a directed graph is a linear ordering of its vertices such that for every edge (u, v) , vertex u appears before v .

In a DAG, DFS finish times naturally produce such an order.

Intuition: if u must come before v , DFS tends to finish u after finishing v .

6.8 Finish Times and Ordering

In a DAG, for every edge (u, v) , DFS guarantees:

$$f[u] > f[v]$$

Therefore, if we list vertices in decreasing order of finish time, every edge points forward in the list.

Topological rule: sort vertices by decreasing finish time.

6.9 DFS-Based Topological Sort

A classical algorithm is:

```
TOPOLOGICAL-SORT(G):  
  
    run DFS(G)  
  
    return vertices sorted in decreasing order of f[u]
```

This works exactly when the graph is acyclic.

If a back edge exists, the graph contains a cycle, and no topological ordering is possible.

6.10 Engineering Use Cases

These two DFS applications appear constantly in practice:

- Cycle detection: validate dependency graphs, prevent circular imports, detect invalid prerequisite loops
- Topological sorting: order build steps, task execution, curriculum dependencies, job pipelines

Takeaway: DFS turns graph structure into actionable ordering information.

6.11 Mini Summary

- Back edges characterize directed cycles.
- No back edge means the graph is a DAG.
- In DAGs, decreasing finish time gives a topological order.
- DFS is therefore a natural tool for dependency analysis.

7 Applications II (SCC & Advanced Structures)

7.1 Applications II

Roadmap (what you will learn in this part):

- What strongly connected components (SCCs) are
- Why DFS finish times matter again
- The transpose graph intuition
- Kosaraju's algorithm
- Why Kosaraju works
- Advanced DFS-based structures and patterns

Depth-First Search becomes especially powerful when we move from traversal to global graph decomposition.

In this part, we study strongly connected components (SCCs) and show how DFS reveals large-scale structure in directed graphs.

7.2 Strongly Connected Components

In a directed graph, two vertices u and v are strongly connected if:

$$u \rightsquigarrow v \quad \text{and} \quad v \rightsquigarrow u$$

That is, each vertex can reach the other.

A strongly connected component is a maximal set of vertices that are pairwise strongly connected.

Interpretation: inside one SCC, every vertex is reachable from every other vertex.

7.3 SCCs as a Higher-Level Structure

If we collapse each SCC into a single super-vertex, we obtain the condensation graph.

A fundamental fact is:

Theorem. The condensation graph of a directed graph is a DAG.

So SCC decomposition turns an arbitrary directed graph into an acyclic structure of components.

7.4 Why Finish Times Matter Again

In Part 6, finish times gave a topological-style order for DAGs. Here they guide us across SCCs.

If one SCC can reach another SCC, then DFS tends to finish the source component later.

High-level idea: decreasing finish times process SCCs in the right structural order.

7.5 The Transpose Graph

The transpose of a directed graph $G = (V, E)$ is the graph $G^T = (V, E^T)$ obtained by reversing every edge:

$$(u, v) \in E \iff (v, u) \in E^T$$

SCCs are preserved under transposition: the components remain the same, but the direction between components reverses.

7.6 Kosaraju's Algorithm

Kosaraju's algorithm computes SCCs using two DFS passes:

1. Run DFS on G and record finish times.
2. Build the transpose graph G^T .
3. Run DFS on G^T in decreasing order of finish times.
4. Each DFS tree in the second pass is one SCC.

Core strategy: first pass computes the right order; second pass extracts the components.

7.7 Kosaraju — Pseudocode Sketch

```
KOSARAJU(G):

    run DFS(G)
    let vertices be ordered by decreasing finish time

    compute G^T

    mark all vertices WHITE

    for each vertex u in decreasing finish-time order:
        if color[u] == WHITE:
            DFS-VISIT(G^T, u)
            output the vertices reached in this DFS as one SCC
```

The second DFS pass groups vertices into strongly connected components.

7.8 Why Kosaraju Works

The first DFS pass identifies an order that respects the component-level structure.

After transposition, edges between SCCs are reversed. So when the second pass starts from the largest remaining finish time, DFS stays inside exactly one SCC before crossing to others.

Key idea: finish times isolate the “right” SCC roots for the transpose graph.

7.9 Complexity

Kosaraju's algorithm runs in linear time:

$$O(|V| + |E|)$$

because:

- the first DFS is linear,
- building the transpose is linear,
- the second DFS is linear.

7.10 Advanced DFS-Based Structures

DFS is also the foundation for deeper graph algorithms and structures:

- Tarjan's SCC algorithm — SCCs using one DFS pass
- Bridges and articulation points — in undirected graphs
- Biconnected components
- Low-link values — a key DFS structural invariant

Takeaway: DFS is not just a traversal algorithm; it is a framework for extracting graph structure.

7.11 Engineering Use Cases

SCC decomposition appears in many real systems:

- dependency cycle analysis
- package/module graph condensation
- compiler optimization passes
- workflow and service dependency analysis

In practice, SCCs help turn cyclic systems into a DAG of strongly connected blocks.

8 DFS vs BFS

8.1 DFS vs BFS

Roadmap (what you will learn in this part):

- The core contrast between DFS and BFS
- Stack vs queue
- What BFS naturally computes
- What DFS naturally computes
- A side-by-side comparison
- When DFS is the right tool
- When BFS is the right tool
- Complexity comparison
- Common misconceptions
- Series summary and closing

DFS and BFS are the two fundamental graph traversal strategies. They both explore graphs in linear time, but they reveal different kinds of structure.

BFS is distance-oriented. DFS is structure-oriented.

Takeaway: If BFS is the language of layers, DFS is the language of recursive structure.

8.2 The Core Difference

The fundamental contrast is:

- BFS explores the graph layer by layer.
- DFS explores one path deeply before backtracking.

This difference changes not only the traversal order, but also the kind of information each algorithm naturally exposes.

8.3 Queue vs Stack

The data structure behind each traversal explains its behavior:

- BFS uses a queue (FIFO).
- DFS uses a stack (explicit or implicit via recursion).

Interpretation: FIFO preserves breadth; LIFO preserves depth.

8.4 What BFS Naturally Computes

BFS is the natural tool when we care about distance from a source.

- shortest paths in unweighted graphs
- layer structure
- minimum number of moves
- reachability by hop count

$$d[v] = \delta(s, v)$$

This is why BFS is the standard algorithm for shortest paths in unweighted graphs.

8.5 What DFS Naturally Computes

DFS is the natural tool when we care about graph structure.

- discovery and finish times
- ancestry relations
- edge classification
- cycle detection
- topological sorting
- strongly connected components

DFS turns traversal into a structural decomposition method.

8.6 Side-by-Side Comparison

Aspect	BFS	DFS
Main strategy	Explore by layers	Explore deeply, then back-track
Primary data structure	Queue	Stack / recursion
Best for	Distance and shortest paths	Structure and decomposition
Shortest path in un-weighted graphs	Yes	No
Cycle detection	Possible, but not natural	Natural and elegant
Topological sorting	No	Yes
Strongly connected components	No	Yes
Time complexity	$O(V + E)$	$O(V + E)$

8.7 When to Prefer DFS

Choose DFS when the problem asks for structural information such as:

- Does the graph contain a cycle?
- Can the graph be topologically sorted?
- What are its strongly connected components?
- Which vertices are ancestors/descendants in the exploration tree?

Rule of thumb: choose DFS when the problem is about the internal organization of the graph.

8.8 When to Prefer BFS

Choose BFS when the problem asks for distance-like information such as:

- What is the shortest path in an unweighted graph?
- What is the minimum number of steps?
- Which vertices lie in layer 1, 2, 3, ... from a source?

Rule of thumb: choose BFS when the problem is about closeness to a source.

8.9 Complexity

With adjacency lists, both traversals run in:

$$O(|V| + |E|)$$

So the main difference is not asymptotic complexity, but the kind of information each traversal exposes.

8.10 Common Misconceptions

- “DFS can also find shortest paths.” Not in general.
- “BFS and DFS are interchangeable.” They are not; they solve different structural goals.
- “DFS is only recursion.” DFS is a traversal principle; recursion is just one implementation style.

8.11 Series Summary

Across this DFS series, we studied:

- the graph model and exploration problem
- deep exploration and backtracking
- the formal DFS algorithm
- discovery and finish times
- edge classification

- cycle detection and topological sorting
- strongly connected components

Final takeaway: DFS is one of the most powerful structural tools in graph theory and algorithms.

Source Pages

- [Overview](#)
- [Part 1](#)
- [Part 2](#)
- [Part 3](#)
- [Part 4](#)
- [Part 5](#)
- [Part 6](#)
- [Part 7](#)
- [Part 8](#)